생산 시퀀스 변화에 대응하는 공장 내 멀티로봇 협업 시스템

Synapse | 2271021 김채영, 2376300 최준희

목차
Table of Contents

01 프로젝트 목표 및 주제

02 시스템 시나리오

03 System Architecture

04 구현 방법

05 결과 및 DEMO

06 어려웠던 점 및 개선 방향

프로젝트 목표 및 주제

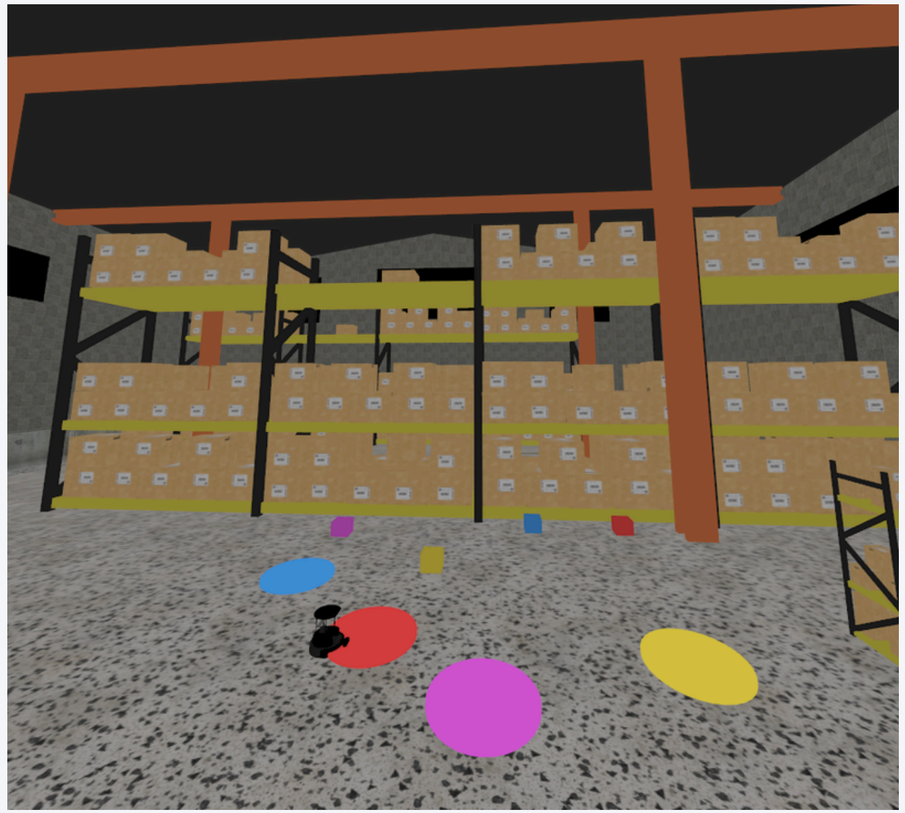
중앙 스케줄러를 통한 다중 로봇·작업 통합 관리

로봇의 위치와 상태를 고려한 운반 작업 자동 배정

작업 우선순위 기반의 효율적인 부품 운반 수행

공장 내 다수의 부품 운반 작업을 로봇의 상태와 작업 우선순위에 따라 중앙 스케줄러가 관리하고 배정

시스템 시나리오



공장 환경에서 2대의 이동 로봇 R1, R2가 부품 박스를 운반

부품 박스는 각각 pickup 위치, drop 위치, 우선순위를 가짐 1

중앙 스케줄러는 현재 로봇의 위치, 상태, 수행 중인 작업을 확인함 2

대기 중인 작업 중 우선순위가 높은 작업을 선택해 사용 가능한 로봇에게 할당 3

로봇은 할당된 박스로 이동한 뒤 pickup하고, 지정된 drop 위치까지 운반 4

작업이 완료되면 로봇은 다시 idle 상태가 되고, 다음 작업을 배정받음 5

System Architecture [1]
[작업 관리 Node]

Keyboard Event Node

키보드 입력을 ROS2 이벤트 메시지로 변환

긴급 작업 추가, 부품 지연, 생산 순서 변경 등의 상황 이벤트 생성

Scheduler Node

전체 작업 큐와 R1·R2의 현재 작업 상태 관리

수신한 이벤트를 작업 큐에 반영

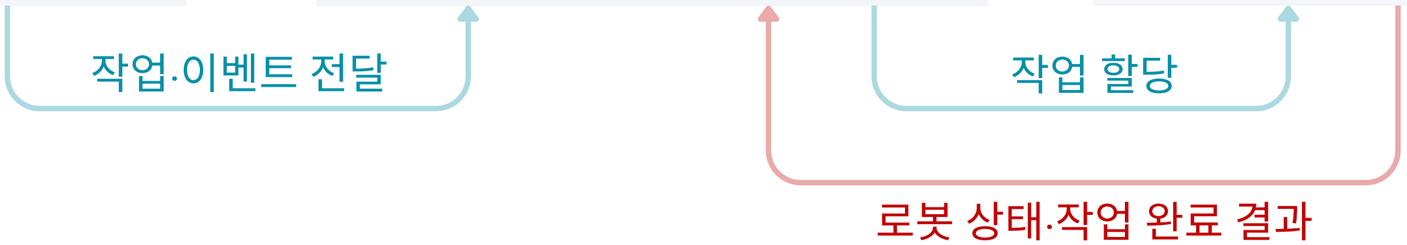
결정된 작업의
Box ID, pickup 위치, drop 위치를 전달

Task Executor Node

Scheduler Node에서 로봇별 작업 정보 수신

pickup 위치를 Nav2 이동 목표로 전달

pickup 완료 후 해당 작업의 drop 위치를 다음 Nav2 이동 목표로 전달



System Architecture [2]

[Robot Node]

Task Executor에서 전달한 pickup·drop 목표를 기반으로
Navigation Node가 이동 경로를 생성하고, Robot Node가 R1·R2의 실제 이동과 상태 갱신을 수행

Navigation Node

- Task Executor Node로부터 pickup·drop **목표 위치 수신**
- 현재 위치에서 목표 위치까지의 **이동 경로 생성**
- 로봇이 목표 위치에 도착할 때까지 주행 명령 전달
- **목표 도착 결과**를 Task Executor Node에 반환

Robot Node

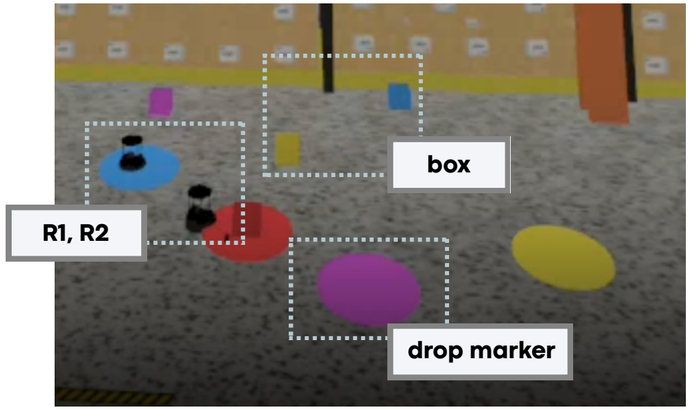
- Navigation Node에서 전달된 이동 명령에 따라 **로봇 구동**
- 로봇별 위치와 상태 정보를 지속적으로 갱신
- pickup·drop 위치 도착 여부를 Navigation Node에 전달
- R1, R2가 서로 **독립적**으로 할당된 운반 작업 수행

System Architecture [3]
[Visualization Node]

로봇의 작업 진행 상황과 박스 운반 상태를 시뮬레이션 화면에서 직관적으로 확인할 수 있도록 구성

Visualization Node

공장 맵 상에 R1, R2, box, drop marker를 시각화

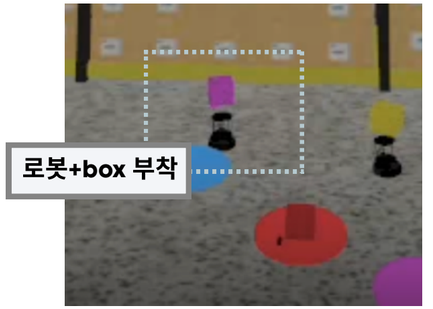


box와 drop marker를 같은 색상으로 표현해 목적지 구분

작업 완료 후 box가 지정된 drop 위치에 놓였는지 확인

Pickup Box Node

로봇이 pickup 위치에 도착하면 해당 box를 로봇에 부착



운반 중 box가 로봇과 함께 이동하도록 상태 갱신

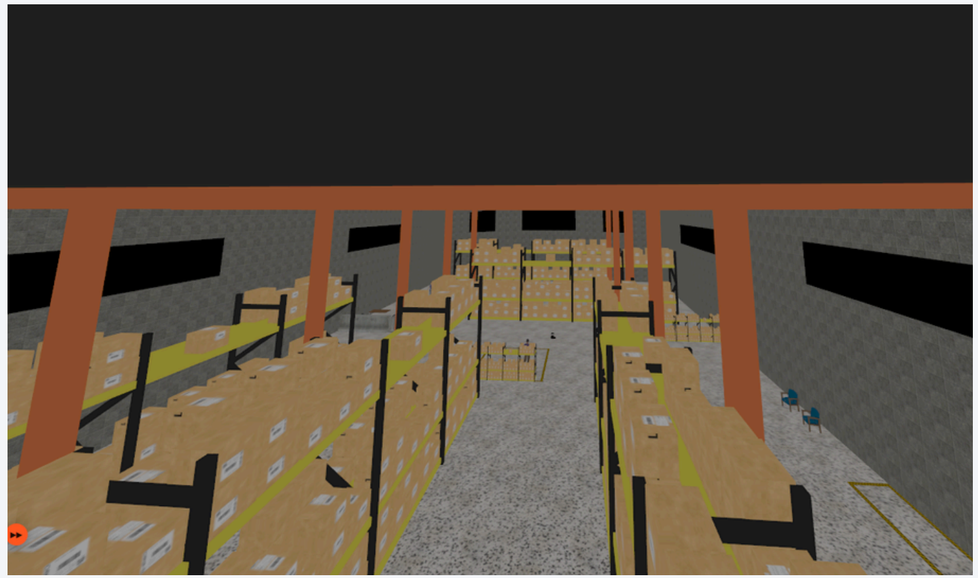
drop 위치 도착 시 box를 로봇에서 분리

box를 지정된 drop marker 위치에 배치하여 작업 완료 처리

구현 방법 1
[환경 및 작업 모델링]

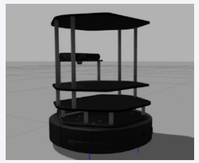
시뮬레이션 환경

AWS RoboMaker의 Small Warehouse World를 활용해 공장 시뮬레이션 환경 구성



작업 모델링

공장 환경에 R1, R2 두 대의 TurtleBot4와 box1~box4 배치



×2

box 1 box 2 box 3 box 4

각 box에 서로 다른 pickup 위치와 drop 위치 지정

각 작업을 Task ID Box ID Pickup 생산 순서 우선 순위 작업 상태 로 관리

작업 상태를 WAITING → ASSIGNED → RUNNING → COMPLETED로 구분

구현 방법 2
[로봇 상태 및 작업 실행]

Task Executor Node는 Scheduler Node로부터 로봇별 작업 정보와 pickup·drop 위치를 수신.
수신한 작업은 다음 실행 단계로 관리:

IDLE → GO_TO_PICKUP → PICKUP → GO_TO_DROP → DROP → IDLE

GO_TO_PICKUP 단계

pickup 위치를 해당 로봇의 Nav2 Goal로 전달

Nav2의 목표 도착 결과를 수신하면 **PICKUP 단계**로 전환하고 carrying=True로 변경

Pickup Box Node는 carrying 상태에 따라 해당 box를 로봇에 부착하여 함께 이동하도록 처리

GO_TO_DROP 단계

지정된 drop 위치를 새로운 Nav2 Goal로 전달

drop 위치 도착 후 **DROP 단계**로 전환하고 carrying=False로 변경

Pickup Box Node는 box를 로봇에서 분리하고 같은 색상의 drop marker 위치에 배치

작업 완료 결과를 Scheduler Node에 전달하고 로봇을 **IDLE 상태**로 변경

구현 방법 3
[작업 배정 및 재스케줄링]

작업의 생산 순서와 준비 상태를 확인한 뒤, 로봇별 예상 비용을 계산하여 비용이 가장 낮은 로봇에게 작업을 배정

1. 이벤트 처리

외부 상황 변화에 따라 **생산 순서와 작업 상태**를
갱신하고 재스케줄링을 요청

이벤트	상태 변경
생산 순서 변경	기존 V1→V2→V3를 V1→V3→V2로 변경
부품 지연	V2 부품을 30초 동안 수행 불가능한 상태로 전환
긴급 작업 삽입	우선순위 10이며 20초 이내 완료가 필요한 URGENT 작업을 새로 생성
작업 선점	긴급 작업 발생 시 pickup 전인 일반 작업은 중단하여 대기 상태로 반환

2, 작업 후보 선별 및 정렬

갱신된 작업 상태를 기준으로 지금 배정할 수 있는
작업을 선별하고 **처리 순서**를 결정

기준	정렬 방식
작업 후보 선별	완료·수행 중인 작업을 제외, 부품이 준비되어 즉시 시작할 수 있는 작업만 선택
작업 순서 결정	긴급 작업을 가장 먼저 배치하고, 일반 작업은 기존 생산 순서에 따라 정렬
동률 기준	같은 생산 단계의 작업은 우선순위가 높은 작업, 완료 목표 시간이 빠른 작업 순으로 정렬

정렬된 작업을 순서대로 R1·R2의 비용 계산 단계에 전달

구현 방법 3
[작업 배정 및 재스케줄링]

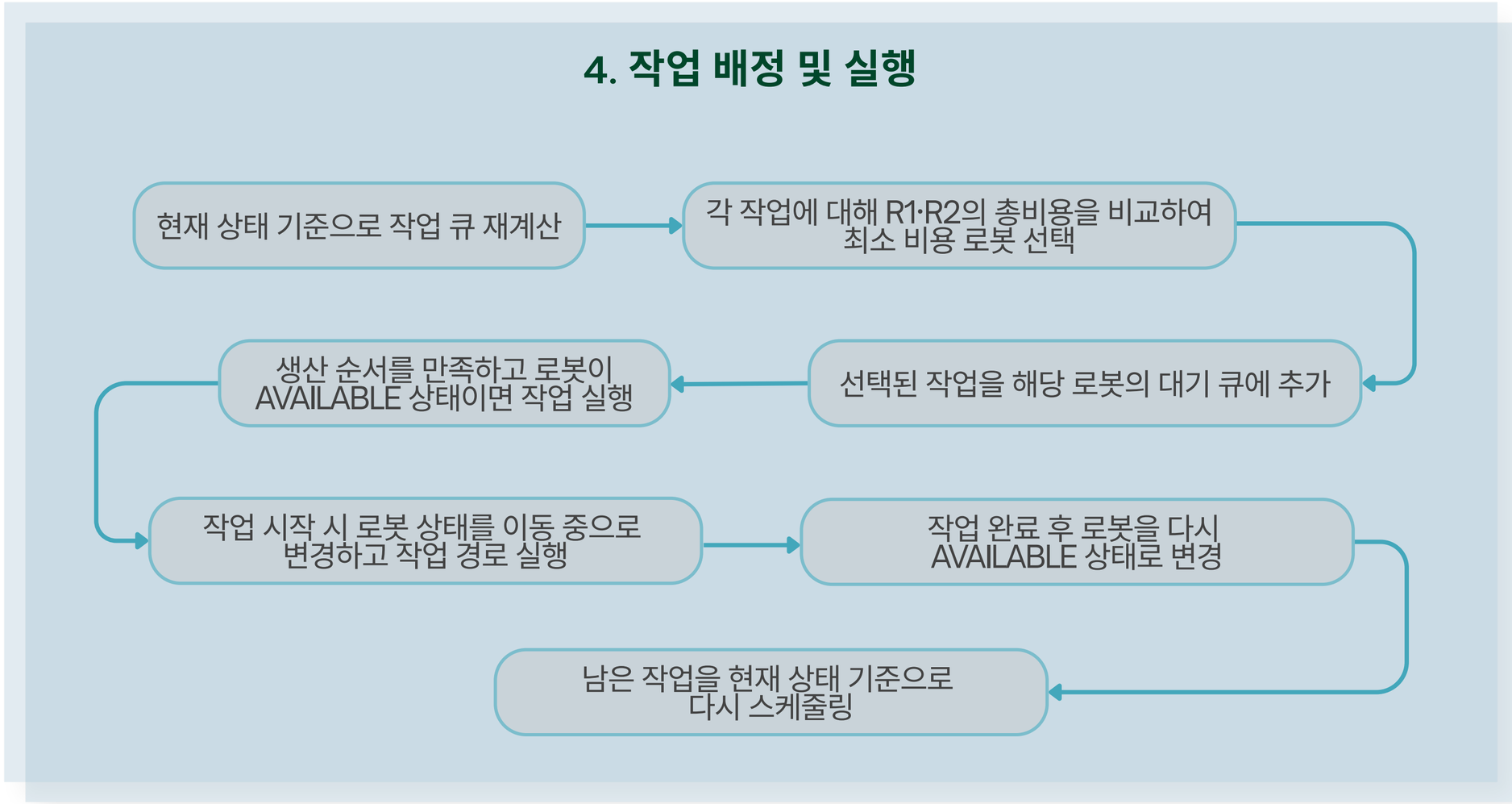
3. 로봇별 작업 비용 계산

각 작업을 R1과 R2가 수행했을 때의 **예상 비용을 계산**하고, **더 적합한 로봇을 선택**

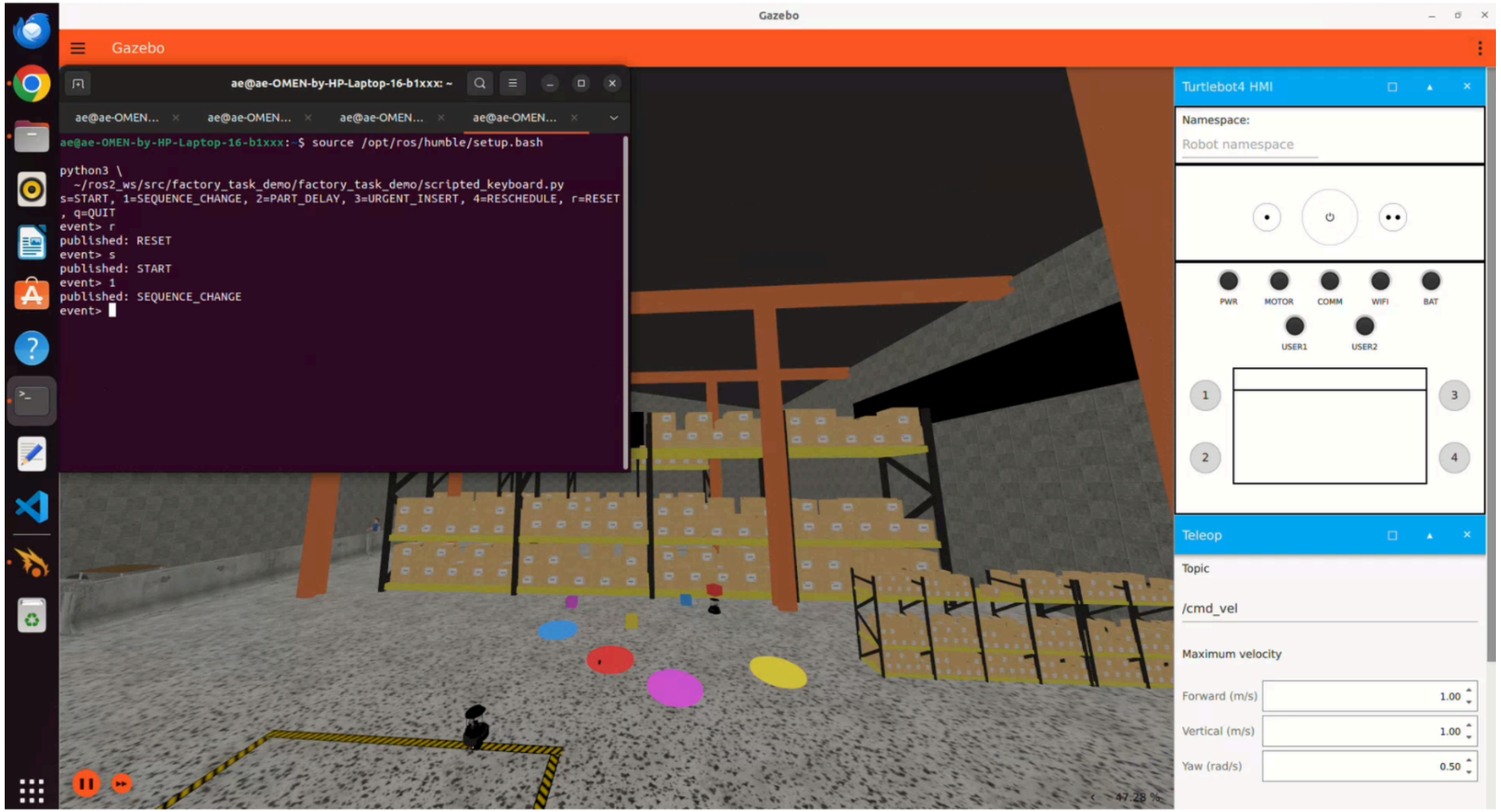
총 cost = 이동 거리 비용 + 대기 큐 비용 + 수행 중 비용 + 완료 지연 비용 + 순서 위반 비용 − 우선순위 조정값

비용 요소	의미	반영 방식
이동 거리 비용	로봇이 박스를 가져와 목적지까지 운반하는 데 필요한 이동량	현재 위치 → pickup → dropoff 거리
대기 큐 비용	로봇에 이미 배정된 작업이 많아 새 작업의 시작이 늦어지는 정도	대기 작업 1개당 1.5 추가
수행 중 비용	현재 다른 작업을 수행 중인 로봇에 추가 작업을 배정하는 부담	수행 중인 작업이 있으면 4.0 추가
완료 지연 비용	예상 완료 시간이 목표 완료 시간을 초과하는 정도	초과 시간 × 1.5 추가
순서 위반 비용	현재 생산 순서보다 뒤의 작업을 먼저 수행할 때 발생하는 불이익	다음 생산 대상과 다르면 8.0 추가
우선순위 조정값	중요도가 높은 작업이 먼저 선택되도록 총비용을 낮추는 값	우선순위 × 1.2만큼 차감

구현 방법 3
[작업 배정 및 재스케줄링]



결과 및 DEMO


Gazebo
Gazebo
ae@ae-OMEN-by-HP-Laptop-16-b1xxx: ~
ae@ae-OMEN...
ae@ae-OMEN...
ae@ae-OMEN...
ae@ae-OMEN...
ae@ae-OMEN-by-HP-Laptop-16-b1xxx:~$ source /opt/ros/humble/setup.bash
python3 \
~/ros2_ws/src/factory_task_demo/factory_task_demo/scripted_keyboard.py
s=START, 1=SEQUENCE_CHANGE, 2=PART_DELAY, 3=URGENT_INSERT, 4=RESCHEDULE, r=RESET
, q=QUIT
event> r
published: RESET
event> s
published: START
event> 1
published: SEQUENCE_CHANGE
event>
Turtlebot4 HMI
Namespace:
Robot namespace
PWR
MOTOR
COMM
WIFI
BAT
USER1
USER2
1
2
3
4
Teleop
Topic
/cmd_vel
Maximum velocity
Forward (m/s)
1.00
Vertical (m/s)
1.00
Yaw (rad/s)
0.50
47.28 %

어려웠던 점 및 개선 방향

좌표계 통일

문제

- 작업 좌표, Nav2 목표 좌표, 로봇 위치가 **서로 다른 좌표계를 사용**하여 pickup·drop 위치가 어긋남
- Gazebo의 박스 위치와 로봇 도착 판정이 일치하지 않는 문제 발생

해결

- pickup·drop 좌표, Nav2 Goal, 거리 계산, 도착 판정을 모두 **map 좌표계로 통일**
- 박스 이동 시 **map 좌표를 Gazebo world 좌표로 변환**

결과

- Scheduler의 작업 좌표와 로봇 이동 목표 일치
- pickup·drop 도착 판정과 박스 부착·배치가 동일한 위치에서 수행됨

로봇과 박스 상태 동기화

문제

- 로봇이 pickup 지점에 도착해도 박스가 자동으로 로봇을 따라 이동하지 않음
- **로봇 상태, 작업 상태, 박스 상태가 서로 다르게 갱신**되면 운반 과정이 화면과 맞지 않는 문제 발생

해결

- pickup 시 **로봇의 carrying 상태와 박스 상태를 동시에** CARRYING으로 변경
- 운반 중 로봇 위치와 방향에 맞춰 박스 위치 지속 갱신
- dropoff 시 박스를 목적지에 배치하고 상태를 DELIVERED로 전환

결과

- 박스가 로봇과 함께 자연스럽게 이동하도록 구현
- Gazebo 화면과 시스템 로그에서 운반 상태를 일관되게 확인

Thank You